

UNIWERSYTET RZESZOWSKI

WYDZIAŁ NAUK ŚCISŁYCH I TECHNICZNYCH

**System Rezerwacji Wizyt i  
Zarządzania Przychodnią  
"ProHealth"**

*Dokumentacja techniczna i analiza architektury backendowej*

**Autorzy:** Kacper Popowicz, Kacper Łazorczyk  
**Numer albumu:** 134963, 134944

**Prowadzący:** Mgr inż. Aleksander Wojtowicz

2025/2026r.

## 1 Kontekst biznesowy systemu

W dobie powszechnej cyfryzacji usług publicznych, sektor ochrony zdrowia staje przed wyzwaniem zapewnienia wysokiej dostępności usług przy zachowaniu najwyższych standardów bezpieczeństwa danych wrażliwych. System "ProHealth" stanowi odpowiedź na te potrzeby, oferując scentralizowaną platformę zarządzania procesami medycznymi.

System integruje trzy kluczowe obszary:

1. **Zarządzanie zasobami ludzkimi:** Ewidencja personelu medycznego (lekarzy) wraz z ich specjalizacjami i indywidualnymi harmonogramami pracy.
2. **Obsługa pacjenta:** Proces od rejestracji tożsamości cyfrowej, przez rezerwację terminów, aż po interakcję z dokumentacją medyczną.
3. **Archiwizacja danych medycznych:** Implementacja trwałej kartoteki pacjenta, dostępnej na podstawie unikalnego identyfikatora PESEL.

## 2 Cele nadrzędne dokumentacji

Niniejsze opracowanie ma na celu dostarczenie kompletnej specyfikacji technicznej backendu aplikacji. Dokumentacja ma służyć zarówno jako podręcznik dla programistów utrzymujących system, jak i podstawa do audytu bezpieczeństwa przesyłu danych medycznych.

## 3 Wykorzystane wzorce i standardy projektowe

Podczas implementacji systemu zdecydowano się na wykorzystanie sprawdzonych wzorców projektowych, co gwarantuje wysoką jakość kodu (*Clean Code*):

- **Dependency Injection (DI):** Wykorzystane do wstrzykiwania zależności takich jak `ApplicationDbContext` czy `ITokenService`, co znacząco ułatwia testy jednostkowe.
- **DTO (Data Transfer Object):** Warstwa separująca modele bazodanowe od kontraktów API, chroniąca przed nadmiarową ekspozycją danych.
- **Static Mapping:** Zastosowanie dedykowanych mapperów statycznych zamiast automatycznych bibliotek, co zwiększa wydajność aplikacji i daje pełną kontrolę nad transformacją danych.
- **Async/Await Pattern:** Wszystkie operacje wejścia/wyjścia na bazie danych są realizowane asynchronicznie, co zapobiega blokowaniu wątków serwera.

Architektura Rozwiązania i Modelowanie Danych

## 4 Struktura logiczna aplikacji

Aplikacja została zbudowana w oparciu o architekturę warstwową, w której każda składowa pełni ściśle określoną rolę.

## 5 Specyfikacja Modelu Relacyjnego

W tej sekcji przedstawiono szczegółową strukturę klas modelu, które mapowane są na tabele bazy danych SQL Server przez Entity Framework Core.

### 5.1 Encja użytkownika systemowego (AppUser)

Model `AppUser` dziedziczy po `IdentityUser`, co integruje system z zaawansowanym mechanizmem *ASP.NET Core Identity*. Pozwala to na obsługę mechanizmów takich jak haszowanie haseł, lockout konta oraz zarządzanie rolami (Admin, Doctor, Patient).

### 5.2 Model Pacjenta i specyfika danych osobowych

Encja `Patient` jest kluczowym elementem systemu, przechowującym dane podlegające szczególnej ochronie.

Tabela 1: Specyfikacja pól modelu Patient

| Nazwa pola | Typ danych  | Opis                             |
|------------|-------------|----------------------------------|
| Id         | int         | Klucz główny (Identity)          |
| FirstName  | string      | Imię pacjenta                    |
| LastName   | string      | Nazwisko pacjenta                |
| Pesel      | string (11) | Unikalny identyfikator biznesowy |
| AccountId  | string (FK) | Powiązanie z kontem Identity     |

```
1 namespace api.Models
2 {
3     public class Patient
4     {
5         public int Id { get; set; }
6         public string FirstName { get; set; } = string.Empty
7         ;
8         public string LastName { get; set; } = string.Empty;
9         public DateTime DateOfBirth { get; set; }
10        public string Pesel { get; set; } = string.Empty;
11        public string PhoneNumber { get; set; } = string.
12        Empty;
13        public List<PatientNotes> Notes { get; set; } = new
14        List<PatientNotes>();
15        public string AccountId { get; set; }
16        public AppUser Account { get; set; }
```

```

14     }
15 }

```

Listing 1: Implementacja modelu Patient

### 5.3 Model Lekarza i Zarządzanie Grafikami

Encja `Doctor` przechowuje informacje o specjalizacji medycznej. W połączeniu z encją `DoctorAvailability` (szczegółowo opisanej w rozdziale o grafiku), tworzy system zarządzania czasem pracy.

```

1 namespace api.Models
2 {
3     public class Doctor
4     {
5         public int Id { get; set; }
6         public string FirstName { get; set; } = string.Empty;
7         public string LastName { get; set; } = string.Empty;
8         public string Specialization { get; set; } = string.
Empty;
9         public string AccountId { get; set; }
10        public AppUser Account { get; set; }
11    }
12 }

```

Listing 2: Implementacja modelu Doctor

### 5.4 Model Wizyty (Appointment)

Wizyta stanowi rekord łączący lekarza i pacjenta w osi czasu.

```

1 namespace api.Models
2 {
3     public class Appointment
4     {
5         public int Id { get; set; }
6         [Required]
7         public string DoctorId { get; set; } // ID z
Identity (AccountId)
8         [Required]
9         public string PatientId { get; set; } // ID z
Identity (AccountId)
10        [Required]
11        public DateTime Date { get; set; }
12        [Required]
13        public string Time { get; set; }
14        public string Status { get; set; } = "Zaplanowana";
15    }

```

16 }

Listing 3: Implementacja modelu Appointment

## 5.5 Notatki i Kartoteka Medyczna

W systemie rozróżniono notatki bieżące (*PatientNotes*) od formalnych wpisów w kartotece (*Kartoteka*).

```

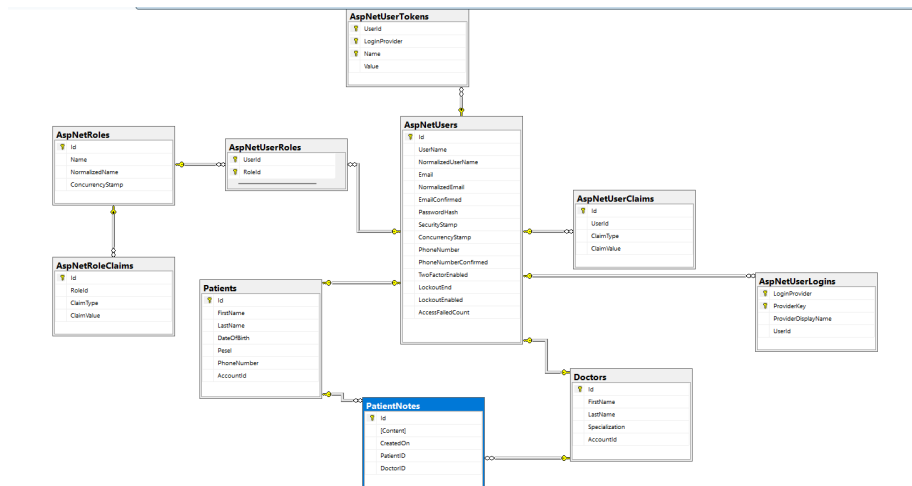
1 public class Kartoteka
2 {
3     public int Id { get; set; }
4     public string Pesel { get; set; } // Klucz wyszukiwania
5     public string Temat { get; set; }
6     public string Opis { get; set; }
7     public DateTime CreatedAt { get; set; } = DateTime.Now;
8 }

```

Listing 4: Implementacja modelu Kartoteka

Infrastruktura Danych i Persystencja

## 6 Diagram ERD Bazy Danych



Rysunek 1: Diagram ERD

## 7 Konfiguracja Contextu Bazy Danych

Sercem warstwy dostępu do danych jest klasa *ApplicationDbContext*, która dziedziczy po *IdentityDbContext<AppUser>*. Wybór tego dziedziczenia jest

kluczowy, ponieważ automatycznie integruje tabele użytkowników, ról i roszczeń (claims) z naszymi autorskimi tabelami medycznymi.

## 7.1 Implementacja klasy Contextu

Poniższy listing przedstawia konfigurację zestawów danych (DbSet) oraz logikę inicjalizacji ról systemowych.

```
1 namespace api.Data
2 {
3     public class ApplicationDBContext : IdentityDbContext<
4         AppUser>
5     {
6         public ApplicationDBContext(DbContextOptions
7             dbContextOptions) : base(dbContextOptions) {}
8
9         public DbSet<Patient> Patients { get; set; }
10        public DbSet<Doctor> Doctors { get; set; }
11        public DbSet<DoctorAvailability> Availabilities {
12            get; set; }
13        public DbSet<Appointment> Appointments { get; set; }
14        public DbSet<PatientNotes> PatientNotes { get; set; }
15    }
16
17    public DbSet<Kartoteka> Kartoteki { get; set; }
18
19    protected override void OnModelCreating(ModelBuilder
20        builder)
21    {
22        base.OnModelCreating(builder);
23        // Konfiguracja ról i relacji Fluent API
24    }
25 }
```

Listing 5: Kod źródłowy ApplicationDBContext.cs

## 8 Mechanizm Seedowania Danych i Role

W celu zapewnienia spójności uprawnień w systemie, proces `OnModelCreating` zawiera logikę "seedowania" ról. Jest to niezbędne dla poprawnego działania mechanizmu RBAC (*Role-Based Access Control*).

Zdefiniowano trzy poziomy uprawnień:

1. **Admin:** Pełny dostęp do zarządzania personelem i konfiguracją systemu.
2. **Doctor:** Dostęp do grafików, historii pacjentów (na podstawie PESEL) oraz edycji kartotek.
3. **Patient:** Dostęp do własnego profilu, rezerwacji wizyt oraz wglądu w osobistą historię badań.

## 9 Modelowanie Relacji Fluent API

W systemie zastosowano podejście *Fluent API* do zdefiniowania relacji jeden-do-jeden pomiędzy tabelą kont (*AspNetUsers*) a profilami medycznymi (*Patients* i *Doctors*). Jest to podejście bezpieczniejsze niż dekorowanie klas atrybutami, ponieważ oddziela logikę bazy danych od modeli domenowych.

```
1 builder.Entity<Patient>()
2     .HasOne(p => p.Account)
3     .WithOne()
4     .HasForeignKey<Patient>(p => p.AccountId);
5
6 builder.Entity<Doctor>()
7     .HasOne(d => d.Account)
8     .WithOne()
9     .HasForeignKey<Doctor>(d => d.AccountId);
```

Listing 6: Konfiguracja relacji jeden-do-jeden

Warstwa Transferu Danych (DTO)

## 10 Rola i znaczenie wzorca DTO

Wzorec *Data Transfer Object* został zaimplementowany w celu oddzielenia wewnętrznej struktury bazy danych od kontraktu publicznego API. W systemie medycznym jest to szczególnie istotne z dwóch powodów:

- **Minimalizacja transferu:** Przesyłane są tylko te dane, które są wymagane przez dany widok (np. brak haszy haseł w liście pacjentów).
- **Bezpieczeństwo:** Zapobieganie atakom typu *Mass Assignment*, gdzie złośliwy użytkownik mógłby próbować nadpisać pola systemowe (np. datę utworzenia konta).

## 11 Szczegółowa specyfikacja obiektów DTO

### 11.1 DTO Rejestracji Pacjenta (RegisterRequestDto)

Poniższy obiekt służy do przechwycenia danych podczas procesu tworzenia konta przez pacjenta. Wykorzystuje on *Data Annotations* do walidacji danych po stronie serwera.

```
1 namespace api.Dtos.Account
2 {
3     public class RegisterRequestDto
4     {
5         [Required]
6         public string? Username { get; set; }
7         [Required]
```

```

8      [EmailAddress]
9      public string? Email { get; set; }
10     [Required]
11     public string? Password { get; set; }
12     [Required]
13     public string? FirstName { get; set; }
14     [Required]
15     public string? LastName { get; set; }
16     [Required]
17     [StringLength(11, MinimumLength = 11)]
18     public string? Pesel { get; set; }
19 }
20 }

```

Listing 7: RegisterRequestDto.cs

## 11.2 DTO Rezerwacji Wizyty (BookAppointmentDto)

Ten uproszczony obiekt transferowy pozwala na przekazanie minimalnej ilości danych potrzebnych do zarezerwowania terminu, co optymalizuje komunikację między frontendem a backendem.

```

1 namespace api.Dtos.Booking
2 {
3     public class BookAppointmentDto
4     {
5         public string DoctorId { get; set; }
6         public string Date { get; set; }
7         public string Time { get; set; }
8     }
9 }

```

Listing 8: BookAppointmentDto.cs

## 11.3 DTO Zarządzania Personalem (RegisterDoctorDto)

W odróżnieniu od pacjenta, rejestracja lekarza wymaga podania specjalizacji medycznej. Obiekt ten jest wykorzystywany w chronionym punkcie końcowym, dostępnym jedynie dla administratorów systemu.

```

1 namespace api.Dtos.Account
2 {
3     public class RegisterDoctorDto
4     {
5         [Required]
6         public string? Username { get; set; }
7         [Required]
8         [EmailAddress]
9         public string? Email { get; set; }
10        [Required]

```



```

11     public string? Password { get; set; }
12     [Required]
13     public string? FirstName { get; set; }
14     [Required]
15     public string? LastName { get; set; }
16     [Required]
17     public string? Specialization { get; set; }
18 }
19 }

```

Listing 9: Specyfikacja RegisterDoctorDto

## 11.4 DTO Kartoteki i Historii Medycznej (KartotekaDto)

Ten obiekt transferowy służy do przekazywania sformalizowanych wpisów medycznych. Zastosowanie prostego obiektu DTO pozwala na szybkie dodawanie rekordów bez konieczności przesyłania metadanych systemowych, takich jak data utworzenia, która jest generowana automatycznie po stronie serwera.

```

1 namespace api.Dtos
2 {
3     public class KartotekaDto
4     {
5         public string Pesel { get; set; }
6         public string Temat { get; set; }
7         public string Opis { get; set; }
8     }
9 }

```

Listing 10: Specyfikacja KartotekaDto

Logika Transformacji - Warstwa Mapperów

## 12 Znaczenie Mapperów w architekturze systemu

Mappery pełnią rolę "tłumaczy" pomiędzy światem bazy danych a światem zewnętrznym (API). Zamiast polegać na automatycznych bibliotekach (takich jak AutoMapper), w projekcie zdecydowano się na autorskie rozwiązanie oparte na **\*\*metodach rozszerzających\*\*** (*Extension Methods*).

Zalety tego podejścia:

- **Wydażność:** Brak narzutu związanego z refleksją (Reflection), co przyspiesza działanie aplikacji.
- **Type-Safety:** Kompilator sprawdza poprawność mapowania już na etapie budowania projektu.
- **Czytelność:** Deweloper ma pełną kontrolę nad tym, które pola są przepisywane.

## 13 Analiza Mapperów Medycznych

### 13.1 DoctorMappers.cs

Mapper ten odpowiada za konwersję modelu lekarza. Zwraca uwagę metoda `ToDoctorFromRegisterDto`, która przyjmuje `accountId` jako osobny parametr, co pozwala na bezpieczne powiązanie lekarza z kontem Identity w trakcie procesu rejestracji.

```
1 namespace api.Mappers
2 {
3     public static class DoctorMappers
4     {
5         public static Doctor ToDoctorFromRegisterDto(this
6 RegisterDoctorDto doctorDto, string accountId)
7         {
8             return new Doctor
9             {
10                 FirstName = doctorDto.FirstName,
11                 LastName = doctorDto.LastName,
12                 Specialization = doctorDto.Specialization,
13                 AccountId = accountId
14             };
15         }
16         public static DoctorDto ToDoctorDto(this Doctor
17 doctorModel)
18         {
19             return new DoctorDto
20             {
21                 Id = doctorModel.Id,
22                 FirstName = doctorModel.FirstName,
23                 LastName = doctorModel.LastName,
24                 Specialization = doctorModel.Specialization
25             };
26         }
27     }
28 }
```

Listing 11: Kod źródłowy DoctorMappers.cs

Usługi i Bezpieczeństwo - TokenService

## 14 Abstrakcja i Interfejsy

W celu zapewnienia testowalności systemu, logika generowania tokenów została odizolowana za interfejsem `ITokenService`. Dzięki temu w testach jednostkowych można łatwo "podstawić" (Mock) tę usługę.

```
1 namespace api.Interfaces
```

```

2 {
3     public interface ITokenService
4     {
5         string CreateToken(AppUser user, IList<string> roles
6         , string pesel);
7     }

```

Listing 12: Interfejs ITokenService.cs

## 15 Szczegółowa analiza procesu generowania JWT

Usługa `TokenService` implementuje standard JWT (*JSON Web Token*). Proces tworzenia tokena obejmuje kilka etapów:

1. **Definicja Roszczeń (Claims):** Do tokena dodawane są informacje o użytkowniku (Email, Name, Role) oraz niestandardowy claim "PESEL".
2. **Podpis Cyfrowy:** Token jest podpisywany kluczem symetrycznym przy użyciu algorytmu **HMAC SHA512**.
3. **Konfiguracja Issuer/Audience:** Zapobiega to użyciu tokena w innym systemie niż "ProHealth".

```

1 public string CreateToken(AppUser user, IList<string> roles,
2     string pesel)
3 {
4     var claims = new List<Claim>
5     {
6         new Claim(JwtRegisteredClaimNames.Email, user.Email)
7         ,
8         new Claim(ClaimTypes.NameIdentifier, user.Id),
9         new Claim("PESEL", pesel ?? ""),
10        new Claim("UserId", user.Id)
11    };
12
13    claims.AddRange(roles.Select(role => new Claim(
14        ClaimTypes.Role, role)));
15
16    var creds = new SigningCredentials(_key,
17        SecurityAlgorithms.HmacSha512Signature);
18    // Dalsza logika deskryptora tokena...
19 }

```

Listing 13: Implementacja TokenService.cs

Implementacja Kontrolerów API

## 16 Architektura Kontrolerów w systemie

Kontrolery w systemie "ProHealth" pełnią rolę koordynatorów. Wykorzystują one wstrzykiwanie zależności (*Dependency Injection*) do komunikacji z bazą danych oraz usługami tokenów. Wszystkie metody są asynchroniczne (`Task<IActionResult>`), co zapewnia skalowalność przy dużej liczbie jednoczesnych pacjentów.

## 17 AccountController - Zarządzanie Tożsamością

Kontroler ten jest odpowiedzialny za procesy uwierzytelniania i autoryzacji. Jest to najbardziej rozbudowany moduł pod kątem logiki bezpieczeństwa.

### 17.1 Endpoint Rejestracji Pacjenta (POST /api/account/register)

Proces rejestracji pacjenta jest wieloetapowy:

1. Walidacja modelu DTO (sprawdzenie poprawności adresu e-mail, długości PESEL).
2. Utworzenie obiektu `AppUser` i zapisanie go w systemie Identity.
3. Automatyczne nadanie roli "Patient".
4. Utworzenie powiązanego rekordu w tabeli `Patients` (mapowanie danych osobowych).

```
1 [HttpPost("register")]
2 public async Task<IActionResult> Register([FromBody]
   RegisterRequestDto registerDto)
3 {
4     if (!ModelState.IsValid) return BadRequest(ModelState);
5
6     var appUser = new AppUser { Username = registerDto.
       Username, Email = registerDto.Email };
7     var createdUser = await _userManager.CreateAsync(appUser
   , registerDto.Password);
8
9     if (createdUser.Succeeded)
10    {
11        await _userManager.AddToRoleAsync(appUser, "Patient
       ");
12        var patientModel = registerDto.
       ToPatientFromRegisterDto(appUser.Id);
13        await _context.Patients.AddAsync(patientModel);
14        await _context.SaveChangesAsync();
15        // Generowanie tokena...
16    }
```

17 }

Listing 14: Fragment logiki rejestracji pacjenta

## 17.2 Endpoint Logowania (POST /api/account/login)

Metoda logowania weryfikuje poświadczenia i zwraca token JWT, który zawiera zaszyte informacje o roli użytkownika oraz jego numerze PESEL.

Tabela 2: Specyfikacja żądania Login

| Parametr | Typ    | Opis                        |
|----------|--------|-----------------------------|
| Username | string | Nazwa użytkownika lub Email |
| Password | string | Hasło (przesyłane w body)   |

## 18 AppointmentController - Logika Rezerwacji

Ten kontroler zarządza interakcją między lekarzem a pacjentem. Zaimplementowano w nim mechanizm "inteligentnego filtrowania" na podstawie roli zalogowanego użytkownika.

### 18.1 Pobieranie wizyt (GET /api/appointment/my-appointments)

Jest to kluczowy punkt końcowy, który demonstruje wykorzystanie Claimów z tokena JWT. System sprawdza rolę użytkownika i na tej podstawie decyduje, czy zwrócić wizyty pacjenta, czy grafik lekarza.

```
1 [HttpGet("my-appointments")]
2 [Authorize]
3 public async Task<IActionResult> GetMyAppointments()
4 {
5     var userId = User.FindFirstValue(ClaimTypes.
        NameIdentifier);
6     var roles = User.FindAll(ClaimTypes.Role).Select(r => r.
        Value).ToList();
7
8     if (roles.Contains("Doctor"))
9     {
10         var appointments = await _context.Appointments
            .Where(a => a.DoctorId == userId)
11             .ToListAsync();
12         return Ok(appointments);
13     }
14     else
15     {
16         var appointments = await _context.Appointments
            .Where(a => a.PatientId == userId)
17             .ToListAsync();
18     }
```

```

19         .ToListAsync();
20         return Ok(appointments);
21     }
22 }

```

Listing 15: Logika filtrowania wizyt na podstawie roli

## 18.2 Statusy wizyt i ich modyfikacja

System obsługuje stany wizyty: *Zaplanowana*, *Zakończona*, *Odwołana*. Zmiana statusu odbywa się poprzez dedykowany endpoint, który sprawdza uprawnienia do modyfikacji danego rekordu (czy pacjent próbuje odwołać własną wizytę, a nie cudzą).

## 19 Obsługa Błędów i Odpowiedzi HTTP

System stosuje ujednolicony standard odpowiedzi:

- **200 OK:** Operacja zakończona sukcesem, dane przesłane w body.
- **201 Created:** Pomyślne utworzenie nowego zasobu (np. nowej wizyty).
- **400 Bad Request:** Błąd walidacji lub brakujące dane.
- **401 Unauthorized:** Brak ważnego tokena JWT.
- **403 Forbidden:** Użytkownik ma token, ale nie posiada wymaganej roli (np. pacjent próbuje dodać lekarza).

Moduł Dokumentacji Medycznej i Historii Chorób

## 20 Rozgraniczenie Notatek a Kartoteki

W architekturze systemu "ProHealth" wprowadzono istotne rozróżnienie pomiędzy dwiema formami gromadzenia danych o stanie zdrowia pacjenta. Decyzja ta została podjęta w celu optymalizacji wydajności bazy danych oraz precyzyjnego zarządzania uprawnieniami.

- **PatientNotes (Notatki):** Są to krótkie wpisy tworzone przez lekarza w trakcie trwania konkretnej wizyty. Są one bezpośrednio powiązane z relacją lekarz-pacjent.
- **Kartoteka (Historia Trwała):** Stanowi nadrzędny zbiór informacji o pacjencie, identyfikowany za pomocą numeru PESEL. Pozwala na dostęp do historii leczenia niezależnie od tego, który lekarz prowadził poprzednie wizyty.

## 21 Analiza kontrolera PatientNotesController

Kontroler ten obsługuje dynamiczne dodawanie uwag do profilu pacjenta. Został on zaprojektowany z wykorzystaniem asynchronicznych metod zapisu, co minimalizuje czas blokowania tabeli podczas wizyt masowych.

```
1 [HttpPost("{patientId}")]
2 [Authorize(Roles = "Doctor")]
3 public async Task<IActionResult> AddNote(int patientId, [
4     FromBody] string content)
5 {
6     var doctorId = User.FindFirstValue("UserId");
7     var note = new PatientNotes
8     {
9         PatientID = patientId,
10        DoctorID = int.Parse(doctorId),
11        Content = content,
12        CreatedOn = DateTime.Now
13    };
14    _context.PatientNotes.Add(note);
15    await _context.SaveChangesAsync();
16    return Ok(new { message = "Notatka dodana pomyślnie" });
17 }
```

Listing 16: Metoda dodawania notatki medycznej

## 22 Moduł Kartoteki i bezpieczeństwo danych (PESEL-based Access)

Unikalną cechą systemu jest sposób autoryzacji dostępu do kartoteki. Zamiast polegać na prostych identyfikatorach bazodanowych, system wykorzystuje zaszyty w tokenie JWT claim "PESEL".

### 22.1 Dostęp Pacjenta (Self-Service)

Pacjent po zalogowaniu ma dostęp jedynie do swojej kartoteki. System pobiera PESEL bezpośrednio z tokena użytkownika, co uniemożliwia podgląd danych innych osób poprzez manipulację parametrami adresu URL (*Insecure Direct Object Reference prevention*).

### 22.2 Dostęp Lekarza (Clinical Access)

Lekarze posiadają uprawnienie do wyszukiwania kartoteki dowolnego pacjenta na podstawie podanego numeru PESEL. Jest to kluczowe w sytuacjach ratunkowych oraz podczas pierwszej wizyty pacjenta u nowego specjalisty.

Tabela 3: Macierz uprawnień do dokumentacji medycznej

| Funkcja                  | Pacjent | Lekarz         | Admin |
|--------------------------|---------|----------------|-------|
| Odczyt własnej kartoteki | TAK     | TAK            | NIE   |
| Odczyt cudzej kartoteki  | NIE     | TAK (po PESEL) | NIE   |
| Dodawanie wpisów         | NIE     | TAK            | NIE   |
| Usuwanie wpisów          | NIE     | NIE            | TAK   |

## 23 Implementacja wyszukiwania w Kartotece

Poniższy fragment kodu prezentuje zaawansowane wykorzystanie LINQ do filtrowania historii medycznej.

```

1 [HttpGet("history/{pesel}")]
2 [Authorize(Roles = "Doctor,Admin")]
3 public async Task<IActionResult> GetHistory(string pesel)
4 {
5     var history = await _context.Kartoteki
6         .Where(k => k.Pesel == pesel)
7         .OrderByDescending(k => k.CreatedAt)
8         .ToListAsync();
9
10    if (history == null) return NotFound("Brak wpis w dla
11        podanego numeru PESEL.");
12    return Ok(history);
13 }
```

Listing 17: Pobieranie historii po numerze PESEL

Konfiguracja Środowiska i Deployment

## 24 Plik konfiguracyjny appsettings.json

Wszystkie wrażliwe dane, takie jak parametry połączenia z bazą danych oraz klucze szyfrujące JWT, znajdują się w pliku konfiguracyjnym. W środowisku produkcyjnym zaleca się przeniesienie tych wartości do zmiennych środowiskowych lub *Azure Key Vault*.

```

1 {
2     "ConnectionStrings": {
3         "DefaultConnection": "Server=localhost,1433;Database=
4         PrzychodniaBD;..."
5     },
6     "JWT": {
7         "Issuer": "http://localhost:5246",
8         "Audience": "http://localhost:5246",
9         "SigningKey": "WielostopniowyKluczSzyfrujacy..."
10    }
11 }
```



10 }

Listing 18: Struktura pliku appsettings.json

Konfiguracja Potoku Przetwarzania i Infrastruktura

## 25 Analiza pliku Program.cs

Plik `Program.cs` stanowi punkt startowy (*Entry Point*) aplikacji. To tutaj definiowane są wszystkie usługi oraz konfigurowany jest potok żądań HTTP (*Middleware pipeline*). W systemie "ProHealth" potok ten został zaprojektowany z myślą o maksymalnym bezpieczeństwie.

### 25.1 Konfiguracja Usług (Dependency Injection)

W pierwszej sekcji pliku następuje rejestracja usług w kontenerze IoC (*Inversion of Control*).

```
1 var builder = WebApplication.CreateBuilder(args);
2
3 // Rejestracja kontroler w i Swaggera
4 builder.Services.AddControllers();
5 builder.Services.AddEndpointsApiExplorer();
6 builder.Services.AddSwaggerGen();
7
8 // Konfiguracja bazy danych
9 builder.Services.AddDbContext<ApplicationDbContext>(options
10     => {
11         options.UseSqlServer(builder.Configuration.
12             GetConnectionString("DefaultConnection"));
13     });
14
15 // Rejestracja us ug autorskich
16 builder.Services.AddScoped<ITokenService, TokenService>();
```

Listing 19: Rejestracja usług systemowych

## 26 Mechanizmy Uwierzytelniania i Autoryzacji

Konfiguracja `AddAuthentication` oraz `AddJwtBearer` to fundament ochrony danych medycznych. System musi wiedzieć, jak zweryfikować klucz cyfrowy przesłany w nagłówku przez klienta.

### 26.1 Parametry walidacji tokena

Wprowadzono rygorystyczne zasady walidacji, takie jak:

- **ValidateIssuer/Audience:** Sprawdzenie, czy token pochodzi z zaufanego źródła.

- **IssuerSigningKey:** Weryfikacja podpisu cyfrowego przy użyciu 256-bitowego klucza.
- **ClockSkew:** Ustawiony na zero, aby zapobiec użyciu tokena nawet sekundy po jego wygaśnięciu.

## 27 Polityka CORS (Cross-Origin Resource Sharing)

Ponieważ backend API komunikuje się z aplikacjami frontendowymi (np. React), konieczne było zdefiniowanie polityki CORS. W obecnej fazie deweloperskiej zastosowano politykę `AllowAll`, jednak w środowisku produkcyjnym zostanie ona ograniczona do konkretnych domen.

```

1 builder.Services.AddCors(options =>
2 {
3     options.AddPolicy("AllowAll", policy =>
4     {
5         policy.AllowAnyOrigin()
6             .AllowAnyMethod()
7             .AllowAnyHeader();
8     });
9 });

```

Listing 20: Konfiguracja polityki CORS

Interaktywna Dokumentacja API (Swagger)

## 28 Standard OpenAPI

Zastosowanie narzędzia Swagger (Swashbuckle) pozwala na automatyczne generowanie dokumentacji w standardzie OpenAPI. Jest to kluczowe dla zespołów frontendowych, ponieważ umożliwia testowanie endpointów "na żywo" bez konieczności używania zewnętrznych narzędzi typu Postman.

## 29 Integracja z JWT w Swaggerze

Dokumentacja została skonfigurowana tak, aby umożliwiać przesyłanie tokena Bearer w nagłówku żądania. Dzięki temu lekarz lub pacjent może zalogować się bezpośrednio w przeglądarce i testować uprawnienia do swojej kartoteki.

Podsumowanie i Perspektywy Rozwoju

## 30 Wnioski techniczne

Zrealizowany system "ProHealth" w pełni realizuje założenia nowoczesnego API medycznego. Dzięki architekturze opartej na DTO, Mapperach oraz silnej typi-

zacji, system jest:

1. **Bezpieczny:** Dzięki claimom w JWT i integracji z ASP.NET Core Identity.
2. **Skalowalny:** Dzięki asynchroniczności i podziale na warstwy.
3. **Łatwy w utrzymaniu:** Dzięki jasnemu podziałowi na modele, kontrolery i usługi.

## 31 Plany rozbudowy (Roadmap)

System stanowi bazę, którą można rozbudować o następujące moduły:

- **Telemedycyna:** Integracja z WebRTC do prowadzenia wideokonsultacji.
- **Powiadomienia:** System Push/SMS informujący o wizytach.
- **AI Diagnostics:** Wstępna analiza notatek medycznych w celu wykrywania trendów chorobowych.

Spis Tabel i Rysunków

### Spis tabel

|   |                                                       |    |
|---|-------------------------------------------------------|----|
| 1 | Specyfikacja pól modelu Patient . . . . .             | 2  |
| 2 | Specyfikacja żądania Login . . . . .                  | 12 |
| 3 | Macierz uprawnień do dokumentacji medycznej . . . . . | 15 |

### Spis rysunków

|   |                       |   |
|---|-----------------------|---|
| 1 | Diagram ERD . . . . . | 4 |
|---|-----------------------|---|